

Q21

*A post-quantum, memory-hard proof-of-work
currency, minable on the machine you already own.*

Absolute cap 21 000 001 Q21	Signatures ML-DSA-87 (FIPS 204)
Block interval 120 s	Proof of work Two-level memory-hard, 2 → 4 GiB
Verification memory 64 → 128 MiB	Governance None

Q21 is a peer-to-peer electronic currency written from scratch. It keeps the foundations that made Bitcoin work — proof of work, unspent outputs, the heaviest chain, every node validating everything — and corrects three decisions that were right in 2008 and have since become dead ends: signatures that a quantum computer will break, a proof of work that only a silicon foundry can win, and a supply ceiling treated as scripture rather than as a parameter. This document describes the protocol as it runs today on the public test network: every number in it is a constant of the consensus code, and every measurement is labelled as measured or as assumed.

Status. Q21 is research code running on a test network. It has not been audited by an independent third party, and its proof of work has not received external cryptanalysis. No Q21 has any value, and none will until this paragraph disappears from this document.

1 What Q21 is, and for whom

1.1 Three debts of 2008

Bitcoin solved a problem nobody had solved: agreeing, without an authority, on who owns what. That part holds, and Q21 does not touch it. Three other things are not design flaws but debts of their era — choices that were correct when made and cannot be repaid inside Bitcoin without breaking it.

What no longer holds	Why it cannot be repaid in place	What Q21 does instead
ECDSA signatures fall to Shor's algorithm on a sufficiently large quantum computer.	The address format never anticipated that another signature scheme could exist; changing it means moving every coin.	The signature scheme is encoded in the address. Today: ML-DSA-87, NIST security level 5.
Mining moved to dedicated circuits and to the few regions with cheap electricity.	SHA-256 is exactly what an application-specific chip does best — a factor of about 10^8 over a processor.	A proof of work bounded by memory bandwidth, with a table that grows so that fixed hardware ages by itself.
The 21 000 000 ceiling arrived by trial and became a totem.	Once sacred, a parameter can no longer be discussed.	21 000 001. The extra unit says the number is a parameter, not a revelation.

1.2 What Q21 keeps

A project that claims to reinvent everything reinvents nothing. Q21 keeps **proof of work** as the mechanism of agreement — not proof of stake, which makes the right to decide depend on what one already owns. It keeps the **UTXO model**: coins, not accounts. It keeps the rule that **the most-worked chain wins**. And it keeps the principle that

every node validates everything itself: a light client asks a server what the chain contains, which means trusting someone in a system built to trust no one.

1.3 Who it is for

Q21 is for people who own an ordinary computer — a laptop with 8 GB of memory, a desktop, a Raspberry Pi — and want to take part in a currency as a peer, not as a customer: to verify it, to mine it without buying special hardware, and to hold it without a custodian. It is for anyone who wants a store of value whose signatures will still be valid after the first cryptographically relevant quantum computer exists. And it is for reviewers and cryptanalysts: the test network is an open invitation to break the design before anything of value depends on it.

Q21 is **not**, today, a payment network for the general public. Post-quantum signatures are large: a simple transaction weighs about 7 361 bytes, thirty times a Bitcoin transaction, and a full 4 MiB block carries about 569 of them — 4.74 transactions per second. That is the throughput of a settlement layer, where sums worth waiting for are moved, not of a wallet used to buy coffee. Section 6 quantifies this; Section 12 names the three ways out and refuses to choose one prematurely.

2 The philosophy, written as rules

Q21's founding sentence is carved into its genesis block: *one voice per machine, not per silicon foundry*. A philosophy that lives only in a preamble is a slogan. Each principle below is paired with the rule of the protocol that embodies it, so that the reader can verify the claim in the code rather than take it on faith.

Principle	How the protocol embodies it
Minalbe by everyone, without a farm.	The proof of work is limited by memory bandwidth, a resource sold on the same market to everyone. A miner needs 2 GiB today and never more than 4 GiB; a machine with 8 GB of RAM suffices for the whole life of the chain (§5).
Verifiable by anyone, forever.	Verifying a block requires only the 64 MiB cache (128 MiB at the ceiling). The cost of mining rises with time; the cost of checking almost does not. A Raspberry Pi validates the chain today and will still do so in year ten (§9).
No owner.	There is no premine. The single unit above 21 000 000 is minted once in the genesis block to an address whose key nobody has — the all-zero hash, spendable only by a preimage attack on SHA-256. It belongs to no one, least of all the person who launched the chain (§3).
No governance.	There is no foundation, no admin key, no checkpoint authority, no signalling mechanism and no upgrade switch. The rules are the code that each node chooses to run. Whoever wants a different rule publishes different code; the network adopts it or does not (§12).
Totally autonomous.	Emission, difficulty, the growth of the mining table, the end of the subsidy at block 26 273 578: everything is a pure function of block height and of what the chain has already recorded. Nothing requires a human decision after launch.
Trust no server.	Bootstrap nodes decide who you talk to first, not what you believe. A snapshot received from a peer is only accepted against a digest the user supplies. The block explorer is served by your own node, from what your machine has validated, and loads no external resource (§8, §9).
Belonging to the people, not to a vendor.	The whole node — consensus, networking, wallet, explorer — is one program with zero mandatory dependencies, compiled by anyone from the public sources, with published checksums. There is nothing to license, nothing to subscribe to, and no service that can be switched off.

3 Money: 21 000 001 and the emission

3.1 Units and the cap

One Q21 is divisible into 10^8 units. The absolute supply is **21 000 001 Q21**: one unit minted once in the genesis block, outside the emission schedule, plus **21 000 000 Q21 created exclusively by mining**. The genesis unit has no technical function. It says one thing: a ceiling must be finite, known in advance and verifiable by anyone — it does not have to be sacred. Two invariants are checked at every block: the sum of unspent outputs never exceeds what has been issued, and what has been issued never exceeds the theoretical schedule at that height.

3.2 The four rules of emission

Bitcoin halves its reward at a stroke every four years; every halving is a cliff — a dated economic shock that invites speculation. Q21 decays continuously and is defined by four rules, all in `emission.rs`, all in integer arithmetic so that the result is reproducible bit for bit on every platform.

```
base(e) = [base(e-1) × 99 715 550 / 100 000 000], base(0) = 13.82743055 Q21, e = [height / 4 320]
```

1. **Continuous decay.** The base reward is multiplied by 0.9971555 every 4 320 blocks (about six days). The cumulative effect is familiar: **the reward halves every 4.01 years** — Bitcoin's rhythm, spread out instead of concentrated on one instant. The factor is stored as an integer fraction, never as a floating-point number.
2. **A slow start.** Over the first 20 000 blocks (about 28 days) the reward rises linearly from zero. Without this ramp, the handful of miners present in the first week would capture a disproportionate share of the total supply. Block 0 issues nothing by mining; the first full reward, 13.67077204 Q21, is paid at block 20 000.
3. **A tail floor.** The reward paid per epoch is $\max(\text{decay}, 0.01 \text{ Q21})$. A geometric decay truncated to the integer never reaches its ceiling — on the raw trajectory, 137 899 Q21 would never have been created. The floor bites only the tail: the decay passes below 0.01 Q21 around year 42, when 99.3 % of the cap is already out.
4. **An exact cap.** Cumulative emission is clipped at the cap, and the subsidy of a block is *defined* as the difference between consecutive cumulative totals. The last issuing block — the 26 273 578th, at 99.91 years — receives the exact remainder of 0.00747908 Q21; after it the subsidy is zero for ever. A test asserts that at that height the total supply is 21 000 001.00000000, not one unit less.

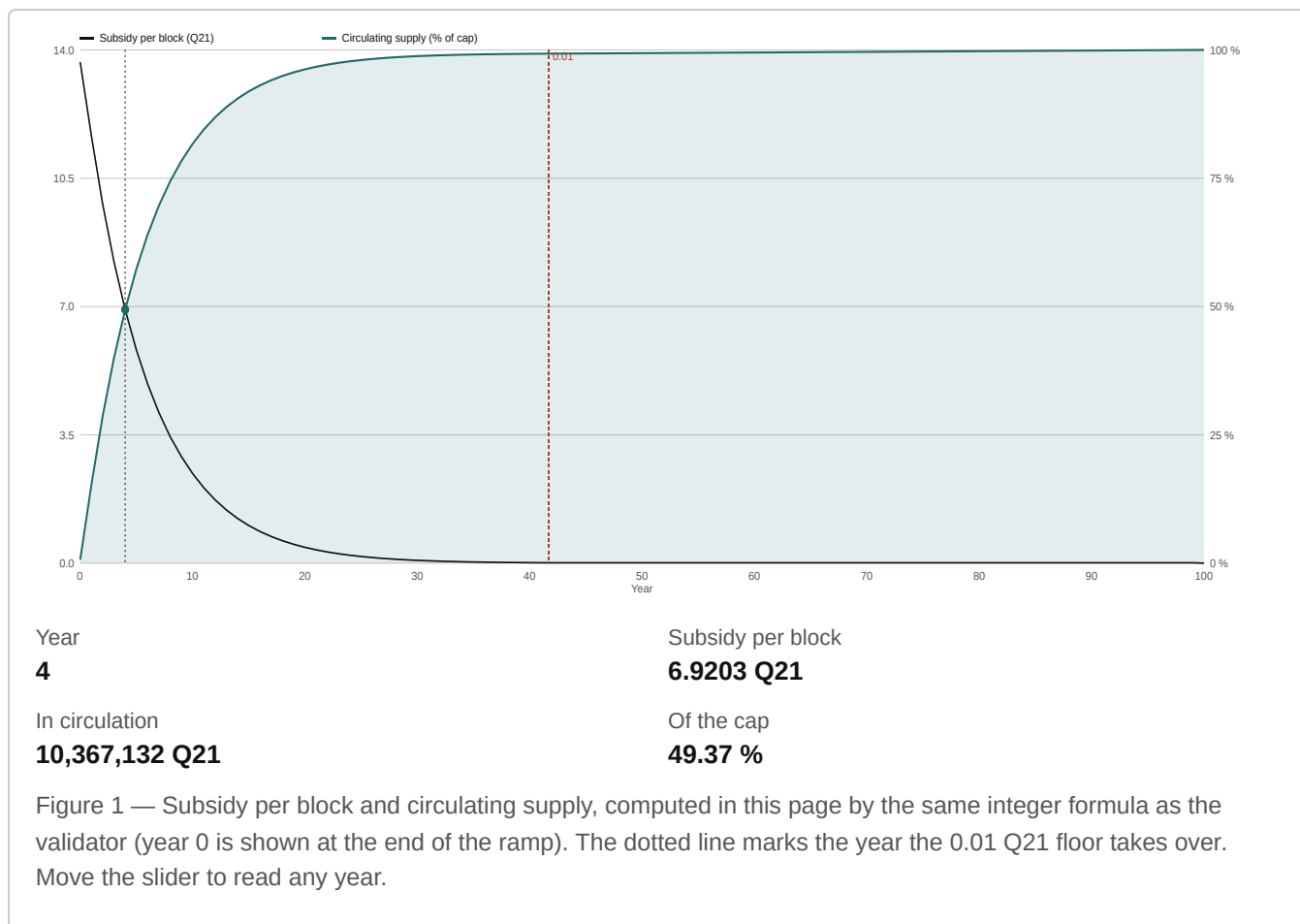


Table 1 — The schedule, year by year (from the consensus functions; one year = 365.25 days = 262 980 blocks)

Year	Per block (Q21)	In circulation (Q21)	Of the cap
1	11.6551	3 205 422	15.26 %
2	9.7961	6 016 430	28.65 %
4	6.9203	10 367 132	49.37 %
8	3.4536	15 617 136	74.37 %
15	1.0263	19 304 086	91.92 %
30	0.0762	20 746 536	98.79 %
50	0.0100	20 868 755	99.38 %
100	0	21 000 001	100.00 %

Table 2 — Thresholds

Milestone	Block	Time after genesis
Half of the cap issued	1 071 360	4.1 years
Three quarters	2 147 040	8.2 years
90 %	3 598 560	13.7 years
99 %	8 605 440	32.7 years
Decay falls below the 0.01 Q21 floor	10 968 480	41.7 years
Cap reached, exactly	26 273 578	99.91 years

A newly mined output can be spent only after **200 confirmations** (coinbase maturity), so that a reorganisation cannot unwind a reward already passed on.

4 Signatures: leaving the prison

ECDSA rests on the hardness of the discrete logarithm. Shor's algorithm solves it in polynomial time on a sufficiently large quantum computer. Nobody knows when such a machine will exist; everybody knows that a chain designed to last cannot bet against it. Bitcoin cannot simply "change scheme": its addresses do not say which scheme they belong to, so adding a post-quantum signature requires a chain break and a migration of every coin.

Q21 encodes the scheme identifier in the address itself. Adding a scheme is a compatible evolution, never a break. Addresses are Bech32m strings with the prefix `q21` on the main network, `tq21` on the test network and `rq21` in regression mode; the encoder is checked against the published BIP-350 vectors.

The scheme in use is **ML-DSA-87** — FIPS 204, NIST security level 5, comparable to AES-256, the highest parameter set the standard defines. Its price is real and is paid deliberately.

Table 3 — Sizes

	ECDSA (secp256k1)	ML-DSA-87
Public key	33 B	2 592 B
Signature	71 B	4 627 B
Resists Shor's algorithm	no	yes

What a signature commits to. The signed digest (tag `Q21/sighash/2`) covers the transaction, the index of the input being signed, the network prefix and the output being spent — its amount and its script. A signature made for the test network is therefore worthless on the main network, and a wallet cannot be tricked into signing for an amount it did not see. The Merkle tree that binds transactions into a block hashes both the transaction identifier and the witness identifier, so that the signatures themselves are committed to by the proof of work.

What Q21 does not write itself. The core does not implement ML-DSA. Writing a lattice signature by hand is professional malpractice: side channels, rejection sampling and constant-time arithmetic are exactly where a home-made implementation looks correct, passes every functional test, and leaks the private key. Q21 defines the interface and links a reviewed implementation, vendored in the repository at a pinned version. The seed is 32 bytes, as FIPS 204 prescribes for all three levels; security is chosen through the parameter level, not through the seed length. A Lamport one-time signature exists for test and regression networks only. The reminder stays useful: in 2022, Rainbow, a NIST finalist, fell in a weekend on a laptop, and SIKE fell in an hour on one core, after five years of public review. This is why the scheme is replaceable by construction.

5 Proof of work: one voice per machine

5.1 The finding that shaped the design

A proof of work whose computation is pure is exactly what a dedicated circuit does best. Q21 uses SHA-256 as a hash function — for identifiers, the Merkle tree, addresses — but the proof of work is something else: a walk through a large table in memory, so that the scarce resource is no longer silicon but **memory bandwidth**, which a chip designer buys on the same market as everyone else.

The first version was wrong, and measurement showed it. Each table element was derived independently, so a miner could recompute elements instead of storing them; on the real 2 GiB table, mining with *no memory at all* cost only 2.86 times more than mining with the whole table. A circuit whose hash is three times faster than a processor had every reason to carry no memory. The property had been "verified" on a 32 MiB table that fitted in cache and displayed a reassuring 6.46×. **Only the measurement at the true size told the truth**, and that lesson is applied everywhere in this document.

5.2 The construction: two levels

The structure follows Ethash. A **cache** of `table/32` — 64 MiB today — is generated as a chain: element i depends on $i-1$, then the whole is mixed three times, so no fragment can be rebuilt without rebuilding everything before it. The **table** — 2 GiB today — derives each element through 256 dependent random accesses into the cache. Each attempt

reads 32 table elements of 32 bytes. Mining without the table no longer saves memory: it multiplies the number of accesses by 256, and therefore the bandwidth.

Table 4 — Relative cost of one attempt as a function of the memory held (measured on the real 2 GiB table)

Fraction of the table held	Memory	First version	Current version
Whole table	2 048 MiB	1.00×	1.00×
One eighth	256 MiB	2.85×	39.7×
None	0 MiB	2.86×	47.8×

The mixing between accesses is full-width: the next index is derived from a 64-bit finaliser over the whole state, and each absorbed element passes through four Feistel rounds. A processor that computes a 32-bit state faster than it reads memory gains nothing, because the state is not 32-bit; the mixing costs about 20 ns against about 100 ns for a random DRAM access, so the walk stays bound by memory.

5.3 The table grows, so hardware ages

The table grows by **5 % per epoch** of 51 200 blocks (about 71 days), from 2 GiB (2^{26} elements) to a ceiling of 4 GiB (2^{27} elements) reached around the third year. A circuit designed around a fixed quantity of memory becomes mediocre as soon as the table outgrows it: dedicated hardware expires on its own, without the defensive hard forks Monero had to inflict on itself four times in 2018–2019. The ceiling was brought down from 8 to 4 GiB because above that the table weighed on the individual — a Raspberry Pi 5 or an 8 GB laptop — more than on dedicated silicon, which buys memory at will.

Table 5 — What a machine must hold

Year	Epoch	Table (miner)	Cache (every node)
0	0	2.00 GiB	64 MiB
1	5	2.55 GiB	82 MiB
2	10	3.26 GiB	104 MiB
3 and beyond	15	4.00 GiB	128 MiB

This asymmetry is deliberate. A node that verifies holds only the cache, and verifying a block costs about 660 μ s instead of 45 μ s — the exact price of the correction, the same trade Ethereum made. A miner rebuilds its table once per epoch: about 24 minutes on two cores, every 71 days. A chain whose verification becomes expensive is a chain nobody verifies any more.

5.4 Difficulty and time

The target adjusts at every block by a linearly weighted moving average over the last **90** blocks, aiming at 120 seconds, with the target never changing by more than a factor of 4 between two blocks. Solve times are clamped asymmetrically: a block may count at most 4 targets early and 6 targets late. A symmetric bound let a miner inject time with a forward-dated block that the honest next block only partly removed, leaving a net difficulty drop and about 33 % extra blocks for half the power; with the asymmetric bound the honest block removes more than the attacker injected, so the manipulation *raises* difficulty and a rational miner gains nothing. A block's timestamp must exceed the median of the previous 11 and may not be more than 600 seconds ahead of the receiving node's clock. The difficulty of a block depends only on its own ancestors, never on the branch a node happens to hold, so two nodes never disagree about the target.

6 Transactions, blocks and fees

A transaction spends outputs and creates outputs; each output carries an amount and a scheme-versioned key hash. Validation checks scripts and amounts before signatures, so that a malformed transaction is rejected before the expensive part. Blocks are produced every **120 seconds**, five times less variance in confirmation time than Bitcoin, and are bounded by size and by weight.

Table 6 — Block and transaction rules

Rule	Value	Why
Maximum block size	4 MiB	1 MiB would hold about 300 transactions, because of signature size.
Target block weight	2 000 000	Signatures (witness) count for a quarter of their bytes; the part that changes the UTXO set is charged in full.
Weight per output	400	Every output must be stored by every node for ever; flooding outputs is charged.
Minimum output	10 000 units (0.0001 Q21)	A consensus dust floor; the genesis unit is exempt. Change below the floor goes to fees.
Minimum fee rate (relay)	10 units per 1 000 weight	Saturating the mempool must cost something; a fee below it is not relayed.
Coinbase maturity	200 blocks	A reward cannot be spent before a deep reorganisation could still remove it.
Merkle leaf	H(txid wtxid)	The proof of work commits to the signatures, not only to the transaction skeleton.

Capacity, stated plainly. With 4 MiB blocks, a full block holds about 569 simple transactions: 4.74 per second, 409 680 per day. Fifteen million holders could each make one transaction every 36 days. Widening the block is brutal: one transaction per person per week for fifteen million people would need 21 MiB blocks and 5.2 TiB of chain per year, which no domestic machine would follow — centralisation by storage, exactly what Bitcoin refused by keeping blocks small. The fee market is rudimentary and has not been observed under load; that is a parameter the real network will settle, not the reasoning.

7 Consensus, forks and finality

The valid chain with the most cumulative work wins. Two mechanisms bound what a reorganisation can do.

A maximum depth. No reorganisation deeper than **720 blocks** (24 hours) is accepted. Beyond that horizon a node considers its history settled: an attacker who mines a competing branch in secret for two days cannot replace a day of the world's transactions.

A rising price for depth. From a depth of 6 blocks, a competing branch must bring **1 % more work per block of depth**, capped at **25 %**. The surcharge applies to the work produced *since the fork point*, on both sides — the only quantity that is disputed, since the common past is contested by no one. The first version applied it to the work since genesis: three months after launch, no reorganisation of depth 7 could succeed even with 100 % of the power, and a twelve-minute network partition produced two permanent chains. The cap was added after a simulation of an ordinary partition — two countries, two providers — mining at equal power on both sides: without it, the two halves became definitive within two hours. With 25 %, any majority above 56 % of the power reunifies the network within the 720-block window, and a deep reorganisation by an attacker still costs a quarter more.

Uncles were tried and retired. Paying a second-place block 25 % of the subsidy — deducted from the miner that included it, so that the cap would not move — meant nobody included one at their own expense; the mechanism only offered an attack surface. A block that carries an uncle is now refused. Compact relay addresses the cause of orphans instead (§8), and at two minutes per block they are rare.

8 The network

The peer-to-peer layer is raw TCP over the standard library, one thread per peer, with no networking library: on code that receives bytes from strangers, every dependency is an attack surface nobody has reviewed. The rule that prevents deadlock fits in one sentence: the shared state lock is never held during a network write, so a slow peer cannot freeze the node. A node keeps up to **32 peers**; the test network listens on port 21121 and identifies itself with its own magic bytes, so that a main-network node and a test-network node cannot even shake hands.

Table 7 — Network mechanisms

Mechanism	What it does
Compact relay	A peer already holds most of a block's transactions in its mempool, so a new block is announced in a few kilobytes and only the missing transactions are requested. Measured on three real nodes: 200 blocks out of 219 announcements relayed without a single round trip. This lowers propagation latency, hence orphans, hence the super-linear advantage of large miners.
Address book against eclipse	Known addresses are stored in buckets by /16 network group, with a salt unique to each node, so that an adversary holding a single address range cannot occupy all of a node's slots. Bootstrap nodes decide who you speak to first, not what you believe.
Liveness	After 45 s of silence a ping is sent; after 100 s without any frame the slot is freed and the bootstraps retried; a loop iteration lasting more than a minute means the machine slept, and all connections are dropped immediately. Measured: a laptop asleep for 75 s recovers 59 blocks within twenty seconds of waking. On a public network the same defect was an eclipse path — opening connections and staying silent occupied every slot.
Misbehaviour score	An unreadable frame costs 10 points, an invalid block 50; at 100 the peer is cut. Message parsing has zero memory amplification: a frame cannot make the receiver allocate more than it sent.
Version gate	Protocol version 2 is both the current and the minimum; a peer announcing an older version is cut at the handshake, so an abandoned rule set cannot linger on the network.
Fast sync	A newcomer may receive, in slices, a snapshot of the UTXO set at height H, the headers from genesis to H and a bounded window of bodies around H. The snapshot carries a MuHash digest — a product in the multiplicative group modulo a 3 072-bit prime, maintained incrementally at every insert and remove — and is accepted only if it matches a digest the user supplies from a source they trust. A peer that sends a snapshot is a convenience, never an authority. Bodies received this way are verified before being written.

9 Running a node

The node is one binary that also contains the wallet and the block explorer. It restarts in seconds from a snapshot of the monetary state — the UTXO set, the tip, the total issued — and replays only the blocks after it; a chain whose nodes cannot be restarted quickly is a chain nobody dares to update. A node that finds a foreign genesis on disk archives that data instead of destroying it.

Table 8 — What a node holds

	Verifying node	Miner	Explorer / archive
Proof-of-work memory	64 → 128 MiB	2 → 4 GiB	64 → 128 MiB
Block bodies on disk	genesis + last 6 008 (pruned)	genesis + last 6 008 (pruned)	all
Disk at full blocks	a few GiB, constant	a few GiB, constant	up to 1.00 TiB per year
Address index	no	no	yes

Pruning. A node that validates for itself needs only what it might still undo — the 1 008-block body window that covers the 720-block reorganisation limit — and what its wallet history displays, 5 000 blocks. With pruning enabled it keeps the genesis and the last 6 008 bodies (about eight days), rewrites the file every 2 016 blocks, and summarises everything older in the snapshot. Three conditions are enforced before a single body is removed: the snapshot has just been written, the header store covers genesis to the snapshot, and every body after the snapshot remains. A pruned node verifies everything it receives; it simply does not keep what it will never read again, so an SD card lasts years. A pruned node refuses to serve as an explorer, because the address index would point to bodies it no longer has.

The explorer. To consult a chain, almost everyone opens a third party's website and thereby trusts a server in a system built to trust no one. Q21's explorer is served by your node on the loopback interface, shows only what your machine validated, loads no font, stylesheet or script from outside — a test asserts the absence of any external URL

— and states in plain words what the protocol does *not* protect. An explorer that shows only what reassures lies by omission.

10 The wallet

The wallet embeds a full node: it does not ask anyone what its balance is, it computes it. Its interface is a page served locally, with the session token carried in the address fragment so that it never reaches a log. Five locks protect it against the web: origin, `Referer` and `Sec-Fetch-Site` checks against a hostile page spending funds unnoticed; a required JSON content type against form-based CSRF, which needs no JavaScript; a `Host` check against DNS rebinding; a token on every RPC method against other programs on the machine; loopback-only listening against the rest of the network. These five locks were tried by 42 real attacks over TCP, each of which was a working exploit before its lock existed.

Table 9 — What protects the keys

Defence	What it prevents
Argon2id sealing of <code>wallet.dat</code> — 64 MiB of memory, 3 passes	A stolen file costs the attacker 64 MiB of memory per password attempt; a graphics card gains almost nothing. Implemented from the standard's reference, checked against the RFC 9106 vectors and an independent implementation; files sealed by the previous key derivation are opened and resealed on first use.
A backup code (seed, Bech32m)	Loss of the machine. The code restores the wallet in any version; the passphrase is confirmed at creation.
Write-ahead of one-time key indices	Signing with a key index that was already used because a crash struck between signing and saving: the wallet records the index, syncs to disk, and only then signs.
Passphrase from the environment read once and erased; core dumps disabled; secrets zeroised on drop	The passphrase or the keys lingering in a process listing, a crash file or freed memory.
Client-side dust and fee checks	Building a transaction the consensus would refuse.

What the wallet still does not do is said as clearly: history is bounded to 5 000 blocks without an address index, and the page says which mode served it; there is no QR code, because the content-security policy forbids external images; the delivered binaries are not code-signed for macOS or Windows, and their SHA-256 checksums are published instead.

11 Threat model, and what is not proven

A security claim is worth what its threat model admits. The table lists the adversaries the design addresses and, for each, the mechanism and its status.

Table 10 — Threats and defences

Threat	Defence	Status
Quantum computer breaks signatures	ML-DSA-87; scheme identifier in the address, so a stronger scheme can be added without a break	STANDARD
Mining captured by dedicated hardware	Memory-bound proof of work, 47.8× cost without memory, table growing to 4 GiB	ASSUMED
Majority attacker rewrites history	720-block horizon; rising surcharge on fork work; 200-block coinbase maturity	MEASURED BY SIMULATION
Timestamp manipulation lowers difficulty	Median of 11; 600 s future bound; asymmetric 4/6 clamp — no strategy lowers difficulty	TESTED
Eclipse of a node	/16 buckets with per-node salt; liveness pings; sleep detection	TESTED
Memory exhaustion by a peer	Zero-amplification parsing; bounded message size; misbehaviour score	MEASURED
Mempool and UTXO flooding	Minimum fee rate; weight per output; dust floor	TESTED
Tampered snapshot	MuHash digest compared with a user-supplied value; bodies verified before write	TESTED
Wallet file theft	Argon2id 64 MiB; zeroisation; no core dumps	VERIFIED AGAINST VECTORS
Hostile web page against the wallet	Five locks, 42 real attacks in the test suite	TESTED
Supply-chain compromise	Zero mandatory dependencies; ML-DSA vendored at a pinned version; pinned toolchain; dependency policy enforced in CI; SHA-256 sums published with every build	IN PLACE
Consensus divergence between platforms	Integer arithmetic only; a Windows PC and a MacBook agreed bit for bit on 438 blocks, each recomputing every signature and proof of work of the other	MEASURED

The test suite counts more than 770 automated checks, including 42 network-level attacks on the wallet, cross-platform agreement, restart from a pruned archive, and the exact equality of the total supply at the last issuing block.

What is assumed, not proven

- **The proof of work has received no external cryptanalysis.** The measurements above were made by its author, on one machine, against a reference implementation — not against an implementation optimised by someone whose job is to beat it. Until that review exists, the anti-ASIC property is a supported hypothesis, not an achievement.
- **Behaviour under a truly hostile network.** The test network exists to observe difficulty with several miners, real reorganisations, and the cost of saturating the mempool when someone actually tries.
- **Centralisation by pools.** At fifteen million miners, a lone participant would find a block every 57 years and would join a pool. The design prevents a chip foundry from taking the network; it does not prevent three or four pools from directing most of the power — which is what happened to Bitcoin, without ASICs to explain it. This is identified, not solved, and must be addressed before a main network.
- **The fee market.** After the subsidy ends, security rests on fees alone. Nobody knows whether fees suffice to pay for the security of a chain in the long run. Bitcoin has the same question and postpones it; Q21 notes it here rather than pretending it does not exist.

12 From the test network to the main network

The main network has not been launched, and this document says what must exist before it is.

Table 11 — Prerequisites for a main network

Prerequisite	Where it stands
A UTXO commitment in the block header	The MuHash accumulator exists and is maintained incrementally; it authenticates snapshots against a user-supplied digest. Committing it in the header would make every snapshot verifiable against the proof of work itself. It is consensus code and will not be rushed the day before a launch.
External cryptanalysis of the proof of work	Open. The test network is the invitation.
An independent human audit of the code	Open. An adversarial internal audit has been made and its findings corrected; it does not replace an external one.
Several independent bootstrap nodes	One public entry point today. A second, held by someone else, comes before anything of value.
Provisional parameters confirmed by measurement	Block interval, block size and the fee market are marked provisional in the code and will be settled by the orphan rate and the usage observed on the test network — by measurement, not by argument.

How change happens without governance

There is no vote, no council and no signalling. A change to the rules is a change to the code; anyone may publish one. A node runs the code its operator chose, and a rule set that the network does not adopt simply does not propagate — a peer on an abandoned protocol version is refused at the handshake. This is slower and rougher than a foundation with a roadmap. It is also the only arrangement in which the currency belongs to the people who run it and to nobody else.

How to take part

Run a node and verify the chain. Mine on the machine you already own — a laptop with 8 GB, a Raspberry Pi with pruning enabled. Read the sources; every mechanism in this document has a test that exercises it. Try to break the proof of work: the property on which the whole project rests is the one that most deserves an adversary. And keep no Q21 as if it had value: on this network it has none, and the network may be reset at any time.

A Parameters at a glance

Parameter	Value	Parameter	Value
Units per Q21	10 ⁸	Absolute supply	21 000 001 Q21
Genesis unit (unspendable)	1 Q21	Mined supply	21 000 000 Q21
Target block interval	120 s	Blocks per year	262 800
Initial base reward	13.82743055 Q21	Decay factor / epoch	0.9971555 / 4 320 blocks
Slow-start ramp	20 000 blocks	Tail floor	0.01 Q21
Last issuing block	26 273 578	Coinbase maturity	200 blocks
Signature scheme	ML-DSA-87	Address encoding	Bech32m, q21 / tq21 / rq21
PoW table, start → ceiling	2 GiB → 4 GiB	PoW cache	table / 32
PoW epoch / growth	51 200 blocks / +5 %	Accesses per attempt / per element	32 / 256
Difficulty window (LWMA)	90 blocks	Max target change per block	4×
Solve-time clamp (early / late)	4 T / 6 T	Median time span / future bound	11 blocks / 600 s
Maximum block size	4 MiB	Target block weight	2 000 000
Witness discount	4	Weight per output	400
Minimum output	10 000 units	Minimum relay fee rate	10 / 1 000 weight
Maximum reorganisation	720 blocks	Reorg surcharge	from depth 6, +1 %/block, cap 25 %
Body window	1 008 blocks	Pruned bodies kept	genesis + 6 008
Peers	32	Protocol version (min)	2 (2)
Wallet sealing	Argon2id, 64 MiB, 3 passes	Test-network port	21121

Q21 takes Bitcoin's foundations, protects them against the quantum computer, and puts mining back within reach of an ordinary machine — saying everywhere what is measured and what is not.

Rei Asanoha

SEPTEMBER 2026

reiasanoha@tutanota.com

Q21 is free software, published under the MIT or Apache-2.0 licence at the recipient's choice. This document describes protocol version 2 as deployed on the public test network in September 2026. Where this document and the consensus code disagree, the code is the protocol. Every release carries a minisign signature whose trusted comment bears this name, and the public key that verifies it is published with the sources. The name signs the work; it decides nothing. Q21 has no governance, and no part of the protocol depends on it. A flaw in the proof of work — or anywhere else in this document — is reported to that address.